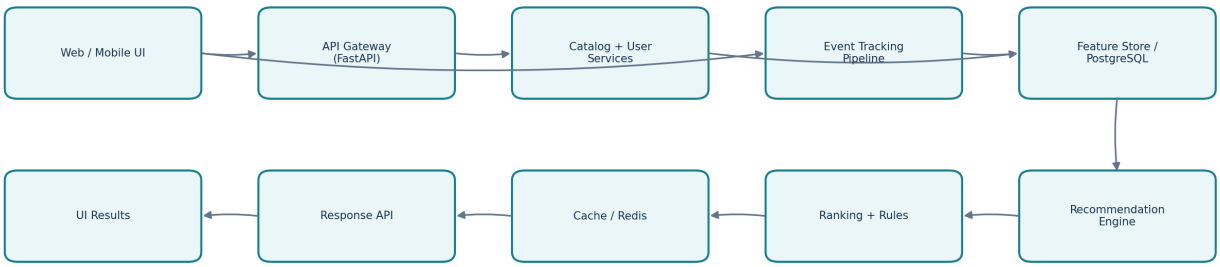
AI-BASED
PRODUCT RECOMMENDATION SYSTEM

Complete Project Documentation & Knowledge Transfer Handbook

10 Members | 8 Months | Python + Machine Learning + React

AI-Based Product Recommendation Platform — High-Level Architecture



Version 1.0 • Planning baseline • October 2026

Document Control

Document	Complete project specification and KT handbook
Project	AI-Based Product Recommendation System
Team	10 members
Timeline	8 months
Primary stack	Python, FastAPI, PostgreSQL, scikit-learn, React
Status	Initial baseline — validate requirements with stakeholders

Contents

• 1. Executive Summary
• 2. Problem Statement
• 3. Objectives and Success Criteria
• 4. Scope
• 5. Personas and Use Cases
• 6. Functional Requirements
• 7. Non-Functional Requirements
• 8. Recommended Technology Stack
• 9. High-Level Architecture
• 10. Detailed Module Catalogue
• 11. Recommendation Strategies
• 12. Proposed Scoring Blueprint
• 13. Database Design and Core Entities
• 14. API Design
• 15. Example Recommendation API Contract
• 16. User Interface Pages and Theme
• 17. Security, Privacy and Responsible Recommendations
• 18. Testing Strategy
• 19. Deployment and Operations
• 20. Eight-Month Roadmap
• 21. Team Allocation and Collaboration
• 22. Sprint and Governance Cadence
• 23. Risks and Mitigations
• 24. Deliverables and Acceptance Checklist
• 25. Knowledge Transfer (KT) Handbook — Required Page Template
• 26. Module-Wise KT Pages
• 27. Suggested Repository Structure
• 28. Sample Demo Script
• 29. Glossary
• 30. Final Handover Checklist

1. Executive Summary

This project builds a production-style product recommendation platform for an e-commerce business. It uses product metadata, browsing events, cart activity, purchases, and contextual signals to recommend relevant items on home pages, product detail pages, search results, and cart pages.

The first release will use a hybrid recommendation strategy: popularity and business rules for cold-start cases, content-based similarity for new or sparse items, collaborative filtering when interaction data is sufficient, and a ranking layer to combine candidate scores with availability and policy rules.

The project is intended as an educational and portfolio-grade system. Use synthetic, anonymized, or explicitly authorized data. Do not expose private user behaviour, and do not present recommendations as guaranteed outcomes.

2. Problem Statement

Many online stores show the same products to every visitor or rely only on manual merchandising. This can make product discovery inefficient, reduce relevance, and make it difficult to measure which recommendation placements are useful.

The system will provide personalized, measurable, and maintainable recommendations while supporting anonymous visitors, new users, new products, changing inventory, and model monitoring.

3. Objectives and Success Criteria

Primary objectives: (1) serve recommendations through documented APIs; (2) support several recommendation strategies; (3) collect product and user interaction events; (4) explain the main factors behind a recommendation where feasible; (5) measure offline model quality and online engagement; (6) deploy a secure, monitored application.

Suggested acceptance targets for the demo: p95 recommendation API latency below 500 ms under the agreed test load; API availability target of 99% during the demo window; no critical/high-severity security issues at release; model evaluation reproducible from a versioned dataset; all core user journeys covered by automated tests. These are project targets, not guaranteed results.

4. Scope

In scope: user accounts and consent-aware profiles; product catalogue and categories; event collection; recommendation candidate generation; ranking and filtering; product search/discovery integration; recommendation placements; admin tools; analytics; API documentation; tests; containerized deployment; monitoring; KT and handover.

Out of scope for the initial release: direct payment processing, real banking/payment credentials, scraping sites without permission, autonomous price changes, and production deployment against a live store without security, privacy, and business approval.

5. Personas and Use Cases

Shopper: browses products, receives recommendations, views details, adds items to cart or wishlist, and can control personalization settings.

Catalog manager: creates and updates product records, categories, images, price, stock status, and product attributes.

Business analyst: reviews impressions, clicks, add-to-cart events, conversion metrics, and recommendation placement performance.

Administrator: manages roles, feature flags, data retention settings, model versions, and system health.

ML engineer: prepares training datasets, evaluates models, registers model versions, and monitors quality and drift.

6. Functional Requirements

FR-01: Users can register, sign in, sign out, and manage profile preferences.

FR-02: Admin users can create, update, deactivate, and import product records.

FR-03: The system validates product identifiers, prices, category references, and stock status.

FR-04: The event collector records consent-appropriate product views, clicks, wishlist actions, cart actions, and purchases.

FR-05: The recommendation API accepts placement, user/session context, optional product ID, page context, and result limit.

FR-06: Candidate generation supports popular, content-based, collaborative-filtering, and rule-based candidates.

FR-07: Ranking removes unavailable, blocked, or duplicate products and returns ordered results with scores/reasons where safe.

FR-08: New or anonymous users receive non-personalized contextual recommendations until enough signals exist.

FR-09: Analysts can review aggregate performance by placement, date range, category, and model version.

FR-10: Administrators can roll back a model version and disable a strategy through configuration.

FR-11: The system provides API docs, structured logs, health checks, and error responses.

FR-12: Users can request deletion of account-linked interaction data according to the defined retention policy.

7. Non-Functional Requirements

Performance: agree load-test assumptions; cache stable candidates; use bounded result limits and pagination for catalogue endpoints.

Security: hash passwords with a modern password-hashing library; use HTTPS in deployed environments; validate all inputs; enforce least privilege; keep secrets outside source control; rate-limit authentication and event endpoints.

Privacy: collect only required event fields; use pseudonymous identifiers; apply consent and retention rules; avoid sensitive attributes in recommendation features; provide deletion/export workflows where required.

Reliability: health/readiness endpoints, database migrations, backups, graceful degradation, and safe fallback recommendations when the model or cache is unavailable.

Maintainability: typed schemas, linting, test coverage for critical logic, code reviews, clear module boundaries, versioned APIs, and documented runbooks.

Observability: request latency, error rates, event ingestion lag, recommendation counts, cache hit ratio, model version, and data/model drift metrics.

8. Recommended Technology Stack

Backend: Python 3.12+, FastAPI, Pydantic, SQLAlchemy, Alembic.

Data and storage: PostgreSQL for core entities; Redis for caching and rate limits; object storage for versioned training datasets and model artefacts.

ML: pandas, NumPy, scikit-learn; implicit or a similar library may be evaluated for implicit-feedback collaborative filtering.

Frontend: React with TypeScript, a component library, and a charting library such as Recharts or Plotly.

Background processing: Celery with Redis for scheduled jobs; Kafka is an optional stretch goal, not a prerequisite.

Testing and delivery: pytest, HTTPX, Playwright, Docker, GitHub Actions, and a cloud platform such as AWS.

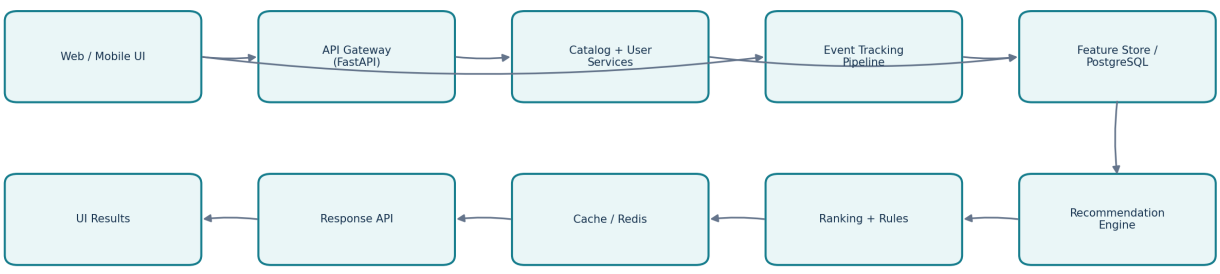
Observability: structured JSON logs, Prometheus/Grafana or a managed equivalent. Select exact versions during Month 1 and pin them in lock files.

9. High-Level Architecture

The UI calls FastAPI endpoints. The API authenticates requests, gathers request context, retrieves candidates from one or more recommenders, applies business filters, ranks results, and returns a stable response schema. Interaction events are stored for analytics and used by scheduled feature engineering and model training jobs.

Keep the online serving path separate from offline model training. Do not train a model inside a user-facing request. Every response should include a request ID and model/strategy version for debugging and measurement.

AI-Based Product Recommendation Platform — High-Level Architecture



10. Detailed Module Catalogue

Module 1 — User Identity, Consent & Profiles — Owner: Member 1

- Registration/login, password reset, profile preferences, guest sessions, consent state, role-based access, account deletion workflow.
- Inputs: email or approved identity provider, preference settings, consent choice. Outputs: authenticated user/session, profile API responses.
- Key tables: users, user_profiles, user_consents, sessions, audit_logs.
- Acceptance: unauthorized requests fail; passwords are never stored in plaintext; consent state is checked before collecting optional behaviour events.

Module 2 — Product Catalogue & Taxonomy — Owner: Member 2

- Product CRUD, categories, brands, attributes, images, price/currency, stock state, product status, bulk CSV import and validation.
- Inputs: admin form or approved catalogue feed. Outputs: normalized searchable product records.
- Key tables: products, categories, product_categories, product_attributes, catalog_import_jobs.
- Acceptance: invalid prices and duplicate external IDs are rejected or reported; inactive/out-of-stock items can be excluded from recommendations.

Module 3 — Behaviour Event Tracking & Data Ingestion — Owner: Member 3

- Track product impressions, views, clicks, wishlist, add-to-cart, remove-from-cart, and purchase events; validate schemas; deduplicate event IDs; batch writes.
- Inputs: event name, timestamp, session/user pseudonym, product ID, placement, request ID, consent flag. Outputs: accepted/rejected event counts and stored events.
- Key tables: events, event_ingestion_errors, sessions. Keep event payloads minimal and avoid unnecessary personal data.
- Acceptance: duplicate event IDs do not inflate counts; invalid product IDs are handled; event ingestion is rate-limited.

Module 4 — Recommendation Candidate Generation — Owner: Member 4

- Generate candidates from trending/popular products, category popularity, content similarity, item-to-item co-occurrence, and collaborative filtering when enough data exists.
- Inputs: user/session context, optional product ID, category, candidate limits. Outputs: candidate product IDs with source and raw score.
- Key artefacts: versioned similarity matrices, popularity aggregates, interaction matrix, candidate-source configuration.
- Acceptance: each strategy has a fallback; unavailable products are filtered; offline evaluation can reproduce candidate sets.

Module 5 — Ranking, Personalization & Explanation — Owner: Member 5

- Combine candidate scores, user affinity, recency, product quality, inventory, diversity and business rules; apply deduplication and top-N ranking.
- Inputs: candidate list, context, feature values and model version. Outputs: ordered products, normalized scores, permitted reason codes.
- Use explainable reason labels such as 'similar to this item' or 'popular in this category' only when supported by actual signals.
- Acceptance: deterministic ranking for a fixed input/model; no unavailable products; configurable diversity and business constraints; safe default if model fails.

Module 6 — Search, Discovery & Recommendation Placements — Owner: Member 6

- Integrate recommendations into home, product detail, category, search-empty, and cart placements; provide product similarity and related-item endpoints.
- Inputs: placement, page context, product/category ID, limit. Outputs: ordered product cards and metadata.
- Define a stable response contract and UI fallback states. Keep search retrieval distinct from recommendation ranking, while allowing the systems to share features.
- Acceptance: placement-specific policies work; empty results are handled gracefully; UI does not block if recommendations time out.

Module 7 — Cart, Wishlist & Shopper Feedback — Owner: Member 7

- Cart and wishlist actions, explicit like/dislike or 'not interested' feedback, save-for-later, and event linkage to recommendation impressions.
- Inputs: authenticated/guest session, product IDs, quantities, feedback. Outputs: cart/wishlist state and feedback events.
- Do not implement payment processing in the MVP. Purchase events can be simulated or imported from an authorized order source.
- Acceptance: users cannot modify another user's cart; duplicate cart lines are handled; feedback affects future recommendations according to documented rules.

Module 8 — Analytics, Experimentation & Reporting — Owner: Member 8

- Track impressions, clicks, add-to-cart, purchases where authorized, click-through rate, conversion rate, coverage, diversity, and offline model metrics.
- Build aggregate dashboards by placement, time, category, and model version; support CSV export and controlled A/B experiments.
- Use consistent event definitions and denominator rules. Avoid exposing small groups or individual-level behavioural details to unauthorized analysts.
- Acceptance: dashboard totals reconcile with test event fixtures; experiment assignment is stable; reports include data freshness and metric definitions.

Module 9 — Data Engineering, Feature Pipeline & Model Lifecycle — Owner: Member 9

- Clean and transform events, build user/item features, split data by time, train models, compare baselines, register model versions, and schedule refresh jobs.
- Use time-based evaluation to reduce leakage. Track dataset version, code commit, feature definitions, model metrics, and training parameters.
- Monitor missing features, item/user coverage, score distribution, data drift and model degradation. Provide rollback to a known-good model.
- Acceptance: training is repeatable; evaluation is documented; model artefacts are versioned; failed jobs alert the team.

Module 10 — QA, Security, DevOps & Release Engineering — Owner: Member 10

- CI/CD, containerization, environment configuration, test automation, API contract tests, load testing, dependency checks, backups and restore drills.
- Create development, staging and demo environments; keep secrets out of Git; document deployment and rollback commands.
- Maintain test reports, defect log, release checklist, monitoring dashboards, incident/runbook notes, and final handover package.
- Acceptance: a clean deployment can be reproduced from the repository; critical journeys pass; backups are verified; no critical/high security findings remain open.

Module 1 — User Identity, Consent & Profiles

Owner: Member 1

- Registration/login, password reset, profile preferences, guest sessions, consent state, role-based access, account deletion workflow.
- Inputs: email or approved identity provider, preference settings, consent choice. Outputs: authenticated user/session, profile API responses.
- Key tables: users, user_profiles, user_consents, sessions, audit_logs.
- Acceptance: unauthorized requests fail; passwords are never stored in plaintext; consent state is checked before collecting optional behaviour events.

Module 2 — Product Catalogue & Taxonomy

Owner: Member 2

- Product CRUD, categories, brands, attributes, images, price/currency, stock state, product status, bulk CSV import and validation.
- Inputs: admin form or approved catalogue feed. Outputs: normalized searchable product records.
- Key tables: products, categories, product_categories, product_attributes, catalog_import_jobs.
- Acceptance: invalid prices and duplicate external IDs are rejected or reported; inactive/out-of-stock items can be excluded from recommendations.

Module 3 — Behaviour Event Tracking & Data Ingestion

Owner: Member 3

- Track product impressions, views, clicks, wishlist, add-to-cart, remove-from-cart, and purchase events; validate schemas; deduplicate event IDs; batch writes.
- Inputs: event name, timestamp, session/user pseudonym, product ID, placement, request ID, consent flag. Outputs: accepted/rejected event counts and stored events.
- Key tables: events, event_ingestion_errors, sessions. Keep event payloads minimal and avoid unnecessary personal data.
- Acceptance: duplicate event IDs do not inflate counts; invalid product IDs are handled; event ingestion is rate-limited.

Module 4 — Recommendation Candidate Generation

Owner: Member 4

- Generate candidates from trending/popular products, category popularity, content similarity, item-to-item co-occurrence, and collaborative filtering when enough data exists.

- Inputs: user/session context, optional product ID, category, candidate limits. Outputs: candidate product IDs with source and raw score.
- Key artefacts: versioned similarity matrices, popularity aggregates, interaction matrix, candidate-source configuration.
- Acceptance: each strategy has a fallback; unavailable products are filtered; offline evaluation can reproduce candidate sets.

Module 5 — Ranking, Personalization & Explanation

Owner: Member 5

- Combine candidate scores, user affinity, recency, product quality, inventory, diversity and business rules; apply deduplication and top-N ranking.
- Inputs: candidate list, context, feature values and model version. Outputs: ordered products, normalized scores, permitted reason codes.
- Use explainable reason labels such as 'similar to this item' or 'popular in this category' only when supported by actual signals.
- Acceptance: deterministic ranking for a fixed input/model; no unavailable products; configurable diversity and business constraints; safe default if model fails.

Module 6 — Search, Discovery & Recommendation Placements

Owner: Member 6

- Integrate recommendations into home, product detail, category, search-empty, and cart placements; provide product similarity and related-item endpoints.
- Inputs: placement, page context, product/category ID, limit. Outputs: ordered product cards and metadata.
- Define a stable response contract and UI fallback states. Keep search retrieval distinct from recommendation ranking, while allowing the systems to share features.
- Acceptance: placement-specific policies work; empty results are handled gracefully; UI does not block if recommendations time out.

Module 7 — Cart, Wishlist & Shopper Feedback

Owner: Member 7

- Cart and wishlist actions, explicit like/dislike or 'not interested' feedback, save-for-later, and event linkage to recommendation impressions.
- Inputs: authenticated/guest session, product IDs, quantities, feedback. Outputs: cart/wishlist state and feedback events.
- Do not implement payment processing in the MVP. Purchase events can be simulated or imported from an authorized order source.
- Acceptance: users cannot modify another user's cart; duplicate cart lines are handled; feedback affects future recommendations according to documented rules.

Module 8 — Analytics, Experimentation & Reporting

Owner: Member 8

- Track impressions, clicks, add-to-cart, purchases where authorized, click-through rate, conversion rate, coverage, diversity, and offline model metrics.
- Build aggregate dashboards by placement, time, category, and model version; support CSV export and controlled A/B experiments.
- Use consistent event definitions and denominator rules. Avoid exposing small groups or individual-level behavioural details to unauthorized analysts.
- Acceptance: dashboard totals reconcile with test event fixtures; experiment assignment is stable; reports include data freshness and metric definitions.

Module 9 — Data Engineering, Feature Pipeline & Model Lifecycle

Owner: Member 9

- Clean and transform events, build user/item features, split data by time, train models, compare baselines, register model versions, and schedule refresh jobs.
- Use time-based evaluation to reduce leakage. Track dataset version, code commit, feature definitions, model metrics, and training parameters.
- Monitor missing features, item/user coverage, score distribution, data drift and model degradation. Provide rollback to a known-good model.
- Acceptance: training is repeatable; evaluation is documented; model artefacts are versioned; failed jobs alert the team.

Module 10 — QA, Security, DevOps & Release Engineering

Owner: Member 10

- CI/CD, containerization, environment configuration, test automation, API contract tests, load testing, dependency checks, backups and restore drills.
- Create development, staging and demo environments; keep secrets out of Git; document deployment and rollback commands.
- Maintain test reports, defect log, release checklist, monitoring dashboards, incident/runbook notes, and final handover package.
- Acceptance: a clean deployment can be reproduced from the repository; critical journeys pass; backups are verified; no critical/high security findings remain open.

11. Recommendation Strategies

Popularity baseline: rank products by recent, smoothed interaction or purchase counts, with minimum data thresholds. Useful for new users and fallback.

Content-based: represent products using category, brand, tags, descriptions or embeddings; compute similarity to the current product or user preference profile. Exclude sensitive or disallowed features.

Collaborative filtering: infer user-item or item-item patterns from interaction events. Use implicit feedback weights such as view < click < wishlist/cart < purchase, but validate weights experimentally.

Hybrid approach: merge candidates from multiple sources, normalize scores, apply hard eligibility filters, then rank with a learned or rule-based scoring layer.

Cold start: new products receive category/content-based exposure; new users receive trending or contextual results; sparse users can optionally set explicit preferences.

Do not rely on offline accuracy alone. Measure coverage, diversity, novelty, calibration, latency, and business outcomes alongside precision/recall or ranking metrics.

12. Proposed Scoring Blueprint

An initial rule-based score can combine: content similarity, collaborative score, category affinity, recency/popularity, availability and diversity. Example weights are configuration placeholders and must be tuned with validation data, not treated as universal best weights.

Hard filters (inactive, unavailable, blocked, duplicate) should be applied regardless of model score. Avoid unfair or sensitive attributes. Keep a baseline recommender available for rollback.

Recommended offline metrics: Precision@K, Recall@K, MAP@K or NDCG@K, catalog coverage, intra-list diversity, and performance segmented by cold-start cohorts.

13. Database Design and Core Entities

users: id, email, password_hash, role, status, created_at. user_profiles: user_id, preference_json, locale, updated_at. user_consents: user_id, purpose, status, captured_at.

products: id, external_id, title, description, category_id, brand, price, currency, stock_status, status, image_url, created_at, updated_at. categories: id, parent_id, name, slug.

events: id, event_id, event_type, user_id nullable, session_id, product_id, placement, occurred_at, request_id, consent_state, metadata_json. Add indexes for occurred_at, product_id, event_type and pseudonymous user/session identifiers.

recommendation_requests: id, request_id, user/session identifier, placement, model_version, latency_ms, created_at. recommendation_impressions: request_id, product_id, rank, score, reason_code.

carts, cart_items, wishlists, wishlist_items: persist shopper state with ownership checks and unique constraints where appropriate. Orders/purchases should only be imported from an authorized source or generated as demo data.

model_registry: model_version, strategy, artifact_uri, dataset_version, metrics_json, status, created_at. experiments: experiment_id, variant, assignment_key, start_at, end_at.

Use UUID or another consistent identifier strategy, foreign keys, timestamps in UTC, migrations, retention jobs, and documented deletion behaviour.

14. API Design

POST /api/v1/auth/register — create account with validation.

POST /api/v1/auth/login — authenticate and issue short-lived access token.

GET /api/v1/products — filter and paginate products.

GET /api/v1/products/{product_id} — retrieve product details.

POST /api/v1/events — ingest a validated interaction event.

GET /api/v1/recommendations?placement=home&limit=10 — return personalized or fallback recommendations.

GET /api/v1/products/{product_id}/similar — related products for a product page.

GET /api/v1/me/wishlist and POST /api/v1/me/wishlist — manage wishlist items.

GET /api/v1/analytics/summary — authorized aggregate analytics.

GET /health and GET /ready — liveness and readiness checks.

Use versioned routes, typed request/response schemas, consistent error objects, pagination, request IDs, rate limits, and explicit authorization. Final contracts must be documented in OpenAPI.

15. Example Recommendation API Contract

Request example: {"placement":"product_detail","product_id":"SKU-1042","session_id":"demo-session-21","limit":8}.

Response example: {"request_id":"req-demo-001","strategy_version":"hybrid-v1","items":[{"product_id":"SKU-202","score":0.87,"reason":"similar_to_current_item"}],"fallback_used":false}.

These are illustrative payloads. Production contracts should not expose internal model details or personal information. Validate limit bounds, product existence, session format and permissions.

16. User Interface Pages and Theme

Recommended visual theme: clean commerce analytics UI with deep navy (#16324F), teal accent (#167D8D), off-white background, restrained borders, readable typography, and accessible contrast.

Shopper pages: home with personalized rails, category listing, product details with related items, cart, wishlist, profile/privacy controls, and feedback controls.

Admin pages: overview KPIs, catalogue management, import validation report, event ingestion status, model registry, strategy feature flags, experiment dashboard, and audit logs.

Analyst pages: recommendation performance, model comparison, data freshness, cold-start coverage, experiment results, and downloadable aggregate reports.

Design states for loading, empty results, unavailable items, API error, stale data, no consent, guest user and small-screen layout. Do not use a chart as the only way to communicate a critical metric.

17. Security, Privacy and Responsible Recommendations

Use HTTPS in deployed environments, password hashing, token expiry/rotation, RBAC, input validation, ORM parameterization, dependency scanning, rate limits, audit logs and secret management.

Do not collect precise location or sensitive personal attributes unless there is a clearly approved and necessary purpose. Do not use protected or sensitive traits to infer product preferences.

Provide clear personalization controls and honour consent and deletion requirements. Define retention windows for raw events, aggregates and model training data.

Prevent catalogue injection and unsafe image URLs; validate imports; limit admin permissions; restrict exports; mask identifiers in logs; protect model artefacts and training data.

Recommendations should not create deceptive dark patterns. Offer non-personalized alternatives and a way to reset preferences. Human review is required for material policy decisions.

18. Testing Strategy

Unit tests: validators, ranking logic, filtering, score combination, consent checks, ownership rules, feature transformations.

Integration tests: database migrations, event ingestion, candidate retrieval, model loading, cache behaviour, and API error contracts.

End-to-end tests: guest home page, signed-in personalized flow, product detail related items, add to wishlist, admin catalogue import, analyst dashboard.

ML tests: data leakage checks, reproducibility, baseline comparison, cold-start cohorts, model artifact compatibility, feature drift and ranking metric calculations.

Performance tests: realistic request mix, concurrency, cache cold/warm behaviour, large catalogues, event bursts and dependency failure.

Security tests: authorization bypass, injection, broken access control, brute-force protection, unsafe imports, secret leakage and dependency vulnerabilities.

19. Deployment and Operations

Use Docker Compose for local development with API, frontend, PostgreSQL and Redis. Use environment variables and a .env.example containing placeholders only.

CI pipeline: lint/type checks, unit tests, integration tests, dependency scan, container build and artifact publishing. Deployment pipeline: staging smoke tests, approval gate, release and rollback.

Maintain migrations, backups, restore verification, health checks, structured logs, alerts and a runbook. Never commit credentials or production datasets.

Operational dashboards should show request volume, p50/p95 latency, error rate, database connections, cache hit ratio, event lag, model version and fallback rate.

20. Eight-Month Roadmap

Month 1 — Discovery and design: personas, scope, requirements, architecture, threat model, UI wireframes, ERD, API contracts, sample dataset and repo conventions.

Month 2 — Platform foundation: authentication, catalogue, database migrations, API scaffolding, frontend shell, CI checks and initial test fixtures.

Month 3 — Event pipeline: event schema, consent handling, ingestion, deduplication, product views/clicks/cart events, catalogue import and analytics definitions.

Month 4 — Baseline recommendations: popularity and content-based recommenders, candidate API, hard filters, offline evaluation and baseline dashboard.

Month 5 — Hybrid personalization: collaborative filtering if data volume permits, ranking layer, cold-start logic, explanation codes, model registry and rollback.

Month 6 — Product integration: home/product/category placements, cart/wishlist feedback, analyst/admin pages, API integration and end-to-end tests.

Month 7 — Hardening: load tests, security checks, privacy deletion workflow, observability, backup/restore drill, staging deployment and bug fixing.

Month 8 — Release and KT: acceptance tests, final tuning, deployment verification, user guide, architecture and operations handover, demo and retrospective.

Month	Milestone
Month 1	Discovery, requirements, architecture, ERD, API contracts
Month 2	Auth, catalogue, migrations, frontend shell, CI
Month 3	Event tracking, ingestion, deduplication, metrics
Month 4	Popularity/content recommenders, offline evaluation
Month 5	Hybrid ranking, cold start, registry, rollback
Month 6	UI placements, cart/wishlist, dashboards, E2E tests
Month 7	Load/security tests, monitoring, backups, staging
Month 8	Acceptance, documentation, KT, final release

21. Team Allocation and Collaboration

Member 1: identity, profiles, consent and RBAC.

Member 2: catalogue, categories and imports.

Member 3: event tracking and ingestion.

Member 4: candidate generation and content-based recommendation.

Member 5: ranking, hybrid scoring and explanation codes.

Member 6: search/discovery placements and recommendation API integration.

Member 7: cart, wishlist and shopper feedback.

Member 8: analytics, experimentation and reporting UI.

Member 9: data engineering, model training, evaluation and registry.

Member 10: QA, security, CI/CD, deployment, monitoring and release management.

Each member should have a primary owner, a backup reviewer, acceptance criteria, unit tests, API contract, documentation, and a demo at each sprint review. Keep module boundaries explicit; avoid making one member the sole owner of all integration knowledge.

22. Sprint and Governance Cadence

Use two-week sprints. Each sprint begins with a prioritized backlog and ends with a demo, tests, updated documentation, and a short retrospective.

Definition of Ready: requirement is understood, dependencies are identified, acceptance criteria exist, and test data is available.

Definition of Done: code reviewed, tests pass, API/schema docs updated, security/privacy checks considered, logs added where needed, and the feature demonstrated in a shared environment.

Maintain a risk register, decision log, issue tracker, release notes, model card, data dictionary and a single source of truth for API contracts.

23. Risks and Mitigations

Insufficient interaction data: start with popularity/content-based approaches and synthetic data; do not promise collaborative-filtering quality before measurement.

Data leakage: use time-based splits and remove future information from training features.

Poor cold-start quality: provide contextual fallback and monitor coverage for new users/items.

Latency spikes: cache candidates, precompute similarities, bound work, and fall back safely.

Privacy risk: minimize event fields, pseudonymize identifiers, define retention and implement deletion workflows.

Scope creep: freeze MVP scope after Month 1 and move optional Kafka/embeddings/advanced experimentation to stretch goals.

Integration delays: publish API contracts early, use mocks, and run integration tests continuously.

24. Deliverables and Acceptance Checklist

Requirements specification, architecture diagram, flowcharts, ERD, UI theme guide and API contract.

Working backend, frontend, database migrations, event pipeline, recommendation strategies, admin/analyst dashboards.

Versioned dataset and model artefacts; evaluation report with baselines and limitations.

Unit/integration/end-to-end test results; performance test summary; security checklist and known-issues log.

Docker setup, CI/CD configuration, deployment guide, monitoring dashboard, backup/restore runbook.

User guide, administrator guide, developer guide, module-wise KT pages, final demo script and handover checklist.

Release acceptance: core journeys pass; no critical/high open security defects; deployment can be repeated; fallback works; metrics definitions are documented; privacy controls are demonstrated.

25. Knowledge Transfer (KT) Handbook — Required Page Template

Each module owner prepares a KT page using this template: module name and owner; purpose; scope; architecture and dependencies; repository path; local setup; environment variables; database tables; API endpoints; input/output examples; business rules; failure modes; test commands; deployment notes; dashboards/logs; known limitations; open issues; and backup owner.

Every KT session should include a live walkthrough, a code walkthrough, a test run, a failure scenario, and a handover Q&A. Record approved internal sessions only, and store recordings in the team's authorized workspace.

26. Module-Wise KT Pages

KT — Module 1 — User Identity, Consent & Profiles

- Primary owner: Member 1. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.

- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Registration/login, password reset, profile preferences, guest sessions, consent state, role-based access, account deletion workflow.
- Inputs: email or approved identity provider, preference settings, consent choice. Outputs: authenticated user/session, profile API responses.
- Key tables: users, user_profiles, user_consents, sessions, audit_logs.
- Acceptance: unauthorized requests fail; passwords are never stored in plaintext; consent state is checked before collecting optional behaviour events.

KT — Module 2 — Product Catalogue & Taxonomy

- Primary owner: Member 2. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Product CRUD, categories, brands, attributes, images, price/currency, stock state, product status, bulk CSV import and validation.
- Inputs: admin form or approved catalogue feed. Outputs: normalized searchable product records.
- Key tables: products, categories, product_categories, product_attributes, catalog_import_jobs.
- Acceptance: invalid prices and duplicate external IDs are rejected or reported; inactive/out-of-stock items can be excluded from recommendations.

KT — Module 3 — Behaviour Event Tracking & Data Ingestion

- Primary owner: Member 3. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Track product impressions, views, clicks, wishlist, add-to-cart, remove-from-cart, and purchase events; validate schemas; deduplicate event IDs; batch writes.
- Inputs: event name, timestamp, session/user pseudonym, product ID, placement, request ID, consent flag. Outputs: accepted/rejected event counts and stored events.
- Key tables: events, event_ingestion_errors, sessions. Keep event payloads minimal and avoid unnecessary personal data.
- Acceptance: duplicate event IDs do not inflate counts; invalid product IDs are handled; event ingestion is rate-limited.

KT — Module 4 — Recommendation Candidate Generation

- Primary owner: Member 4. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.

- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Generate candidates from trending/popular products, category popularity, content similarity, item-to-item co-occurrence, and collaborative filtering when enough data exists.
- Inputs: user/session context, optional product ID, category, candidate limits. Outputs: candidate product IDs with source and raw score.
- Key artefacts: versioned similarity matrices, popularity aggregates, interaction matrix, candidate-source configuration.
- Acceptance: each strategy has a fallback; unavailable products are filtered; offline evaluation can reproduce candidate sets.

KT — Module 5 — Ranking, Personalization & Explanation

- Primary owner: Member 5. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Combine candidate scores, user affinity, recency, product quality, inventory, diversity and business rules; apply deduplication and top-N ranking.
- Inputs: candidate list, context, feature values and model version. Outputs: ordered products, normalized scores, permitted reason codes.
- Use explainable reason labels such as 'similar to this item' or 'popular in this category' only when supported by actual signals.
- Acceptance: deterministic ranking for a fixed input/model; no unavailable products; configurable diversity and business constraints; safe default if model fails.

KT — Module 6 — Search, Discovery & Recommendation Placements

- Primary owner: Member 6. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Integrate recommendations into home, product detail, category, search-empty, and cart placements; provide product similarity and related-item endpoints.
- Inputs: placement, page context, product/category ID, limit. Outputs: ordered product cards and metadata.
- Define a stable response contract and UI fallback states. Keep search retrieval distinct from recommendation ranking, while allowing the systems to share features.
- Acceptance: placement-specific policies work; empty results are handled gracefully; UI does not block if recommendations time out.

KT — Module 7 — Cart, Wishlist & Shopper Feedback

- Primary owner: Member 7. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.

- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Cart and wishlist actions, explicit like/dislike or 'not interested' feedback, save-for-later, and event linkage to recommendation impressions.
- Inputs: authenticated/guest session, product IDs, quantities, feedback. Outputs: cart/wishlist state and feedback events.
- Do not implement payment processing in the MVP. Purchase events can be simulated or imported from an authorized order source.
- Acceptance: users cannot modify another user's cart; duplicate cart lines are handled; feedback affects future recommendations according to documented rules.

KT — Module 8 — Analytics, Experimentation & Reporting

- Primary owner: Member 8. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Track impressions, clicks, add-to-cart, purchases where authorized, click-through rate, conversion rate, coverage, diversity, and offline model metrics.
- Build aggregate dashboards by placement, time, category, and model version; support CSV export and controlled A/B experiments.
- Use consistent event definitions and denominator rules. Avoid exposing small groups or individual-level behavioural details to unauthorized analysts.
- Acceptance: dashboard totals reconcile with test event fixtures; experiment assignment is stable; reports include data freshness and metric definitions.

KT — Module 9 — Data Engineering, Feature Pipeline & Model Lifecycle

- Primary owner: Member 9. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.
- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- Clean and transform events, build user/item features, split data by time, train models, compare baselines, register model versions, and schedule refresh jobs.
- Use time-based evaluation to reduce leakage. Track dataset version, code commit, feature definitions, model metrics, and training parameters.
- Monitor missing features, item/user coverage, score distribution, data drift and model degradation. Provide rollback to a known-good model.
- Acceptance: training is repeatable; evaluation is documented; model artefacts are versioned; failed jobs alert the team.

KT — Module 10 — QA, Security, DevOps & Release Engineering

- Primary owner: Member 10. Backup reviewer: assign during Month 1.
- Purpose: explain the module's responsibility, boundaries, dependencies, and operational behaviour.
- Repository path: to be finalized in the team repository; recommended folder name based on the module slug.
- Local setup: install pinned dependencies, configure a local environment file from .env.example, start PostgreSQL/Redis if needed, run migrations, then run module tests.
- Walkthrough checklist: show the main entry point; explain request/data flow; demonstrate a successful case and a failure case; inspect logs; run tests; explain rollback or fallback behaviour.

- Handover checklist: README updated; API/data contracts linked; tests green; known limitations listed; owner and backup identified; no secrets or personal data in examples.
- CI/CD, containerization, environment configuration, test automation, API contract tests, load testing, dependency checks, backups and restore drills.
- Create development, staging and demo environments; keep secrets out of Git; document deployment and rollback commands.
- Maintain test reports, defect log, release checklist, monitoring dashboards, incident/runbook notes, and final handover package.
- Acceptance: a clean deployment can be reproduced from the repository; critical journeys pass; backups are verified; no critical/high security findings remain open.

27. Suggested Repository Structure

```
recommendation-platform/
backend/app/main.py, api/, core/, models/, schemas/, services/, recommenders/, repositories/, workers/
frontend/src/pages/, components/, api/, hooks/, styles/
ml/data_prep/, features/, training/, evaluation/, registry/, artifacts/ (do not commit large/private data)
tests/unit/, tests/integration/, tests/e2e/
infra/docker/, ci/, monitoring/, migrations/
docs/requirements.md, architecture.md, api-contracts.md, data-dictionary.md, model-card.md, runbooks/, kt/
docker-compose.yml, .env.example, README.md, pyproject.toml, lock file
```

Adapt this structure to the team's framework choices; keep model training and online serving code separated.

28. Sample Demo Script

1. Sign in as a demo shopper and open the home page.
2. View a product and inspect the related-products rail.
3. Click a recommendation and add it to wishlist/cart; verify event logging.
4. Open the analyst dashboard and show impressions, clicks, and data freshness.
5. Change a demo product to inactive and confirm it is filtered from recommendations.
6. Switch to a fallback strategy or prior model version and show safe behaviour.
7. Sign in as an admin and review catalogue validation errors and audit logs.
8. Show automated test results, health endpoint, model version, monitoring view, and deployment runbook.

29. Glossary

Candidate generation: retrieving a pool of products that might be relevant before final ranking.

Ranking: ordering candidate products by estimated relevance and applicable constraints.

Cold start: limited or absent interaction history for a new user or product.

Implicit feedback: behaviour such as views, clicks, carts and purchases used as indirect preference signals.

CTR: click-through rate, typically recommendation clicks divided by recommendation impressions under a documented attribution rule.

Coverage: proportion of users or catalog items for which the recommender can provide results.

Data leakage: training or evaluation using information that would not be available at prediction time.

Model drift: a change in data patterns or performance that can make a deployed model less reliable.

Fallback: a simpler known-good recommendation strategy used when personalization is unavailable or unsuitable.

30. Final Handover Checklist

Repository access and branch protections confirmed; all work merged and tagged.

README, setup guide, environment template and dependency lock file present.

Database migrations and sample synthetic data included; backups and restore instructions verified.

API docs, ERD, architecture diagrams, data dictionary and model card delivered.

All KT sessions completed; primary and backup owners assigned; open issues accepted by the receiving team.

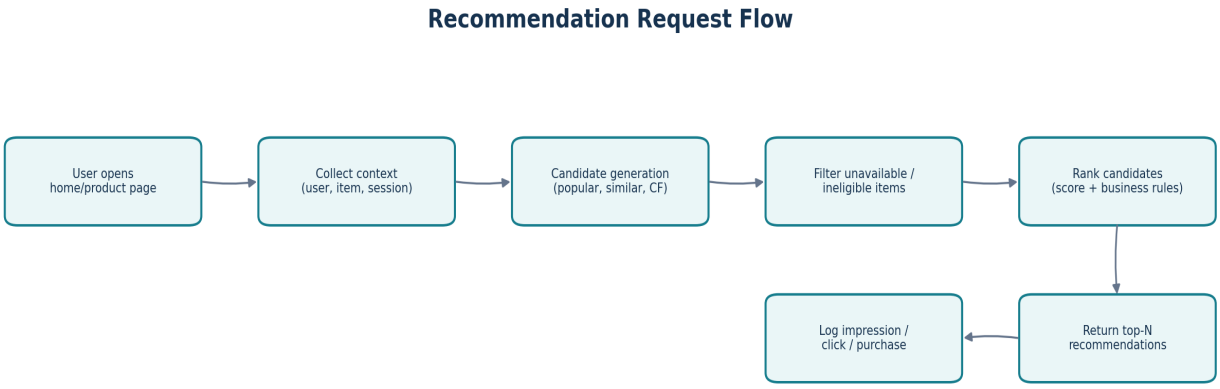
Deployment and rollback tested; monitoring and alert ownership defined.

Privacy, retention, consent, account deletion, and access control demonstrated.

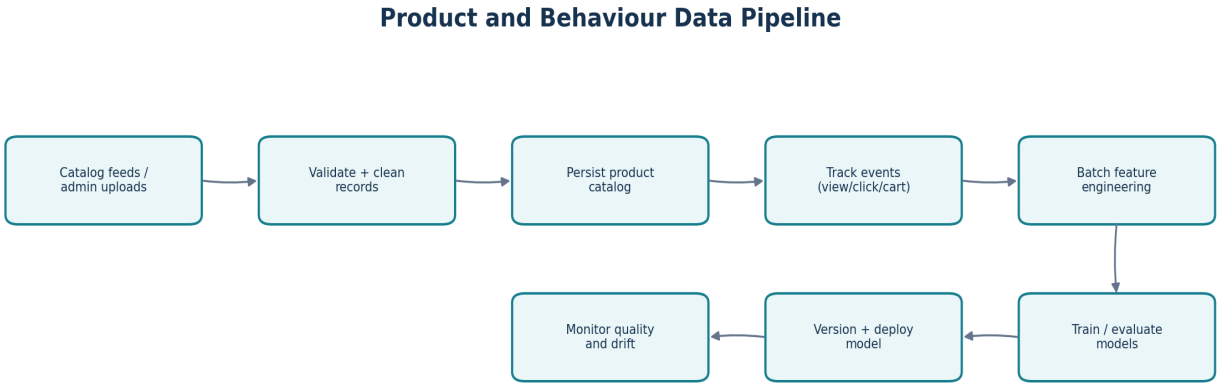
Final report, demo slides, test summary, known limitations and future enhancements delivered.

Appendix A — Flowcharts and Data Diagrams

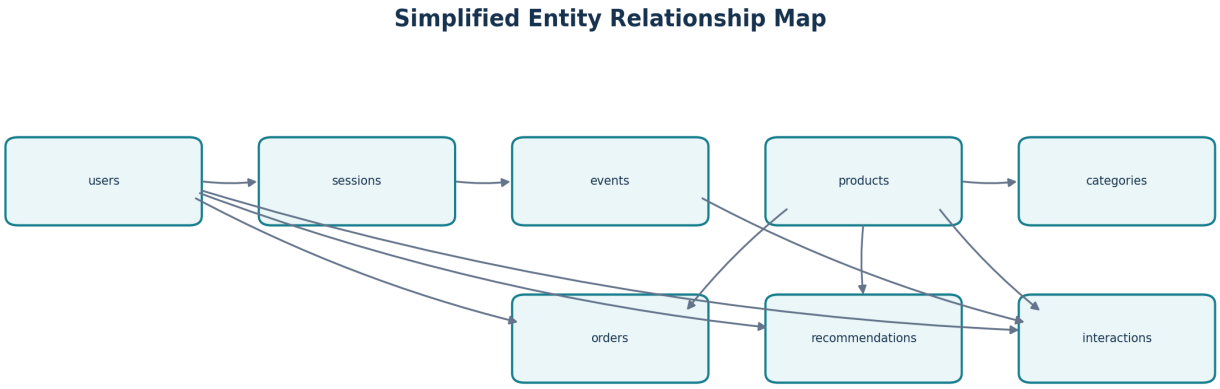
Recommendation request flow



Product and behaviour data pipeline



Simplified entity relationship map



Appendix B — Suggested Sprint Review Template

- Sprint goal and completed stories
- Demo links and environment
- Test results and coverage summary

- API/schema changes
- Risks, blockers and decisions
- Security/privacy considerations
- Next sprint goals and owners